

Second Lab Assignment: Cache Simulator - Report

Francisco Fonseca - 102492; Gonalo Rua - 102604; Joo Gouveia - 102611

4.1 Directly-Mapped L1 Cache

For the first task we started by changing the way we calculate the offset (6 bits), index (8 bits) and tag (18 bits). We also changed the structure Cache so that we have multiple lines (array of size L1_BLOCK_COUNT). To access the right location inside the L1Cache array, we calculated the physical address of the corresponding block, multiplying the index with the block size. Finally, to access DRAM in case of miss with dirty flag, we had to calculate the physical address of the previous tag in that block on the DRAM array. The rest of the code remained practically unchanged.

4.2 Directly-Mapped L2 Cache

To accomplish the second task we reused the code of the Directly-Mapped L1 Cache, since the logic of the Directly-Mapped L2 Cache is almost the same, just changing the size of the index (to 9 bits since the cache size is double the size). Also we had to change the L1 Cache code for when a miss occurs it reads from the L2 Cache and when there is a miss with dirty flag we write to the L2 Cache.

4.3 2-Way L2 Cache

To add 2-way associativity to the L2 cach we only had to change the L2 cache code, while the first level cache remained unchanged. We created the struct CacheSet, which contains one array of type CacheLine. The L2 cache struct now stores one array of type CacheSet instead of CachLine. When we intend to load a block to a full set, we use the LRU policy to decide which blocks is replaced.